Laboratorio 05

Segnali e processi in C: itercounter, figliCounter, cercaCarFile

Questo documento spiega i file della cartella **Lab05-soluzioni**: la struttura dei programmi, il significato di ogni parte, cosa fanno e come rifare gli esercizi da zero in vista dell'esame.

File	Esercizio	Idea in una riga
Lab05-es01-itercounter.c	Es. 1	N figli contano i secondi; dopo Nsec il padre li ferma con SIGUSR1.
Lab05-es02-figliCounter.c	Es. 2	Il padre manda a caso SIGUSR1/SIGUSR2 ai figli, che incrementano/decrementano un contatore; CTRL-C chiude tutto.
Lab05-es03-cercaCarFile-signal.c	Es. 3	Gara tra figli che contano un carattere in un file (versione con signal()).
Lab05-es03-cercaCarFile-sigaction.c	Es. 3	Stessa gara, versione con sigaction().

1. Concetti comuni

Concetto	Significato
`fork()`	Duplica il processo. Restituisce 0 nel figlio, il PID del figlio nel padre, valore negativo in caso di errore.
Segnale	Notifica asincrona inviata a un processo (SIGUSR1, SIGUSR2, SIGINT...).
`kill(pid, sig)`	Invia *sig* al processo *pid*. Con *pid == 0* lo invia a tutto il gruppo di processi, **padre incluso**.
Handler	Funzione `void f(int signo)` eseguita quando il segnale arriva.
`sigaction / signal`	Installano l'handler. sigaction e' la versione moderna e portabile.
`SIG_IGN`	Disposizione che significa "ignora il segnale".
`pause()`	Il processo dorme finche' non arriva un segnale e il relativo handler termina.
`sleep(n)`	Dorme n secondi, ma viene **interrotto** da un segnale gestito.
`wait(&status)`	Il padre attende la terminazione di un figlio; va chiamata una volta per figlio.
`static volatile sig_atomic_t`	Tipo corretto per variabili condivise tra codice normale e handler. *volatile*: il compilatore non la tiene in un registro. *sig_atomic_t*: lettura/scrittura atomiche.

Regola fondamentale: gli handler si installano **prima della fork()**, perche' i figli ereditano handler e disposizioni. Dopo la fork, padre e figli possono cambiare le proprie disposizioni senza influenzarsi a vicenda.

2. Esercizio 1: itercounter

Uso: `./itercounter <num_processi> <num_secondi>`

Cosa fa. Il padre crea N figli. Ogni figlio, ogni secondo, incrementa un contatore. Dopo Nsec secondi il padre invia SIGUSR1 a tutti i figli; ciascuno stampa il numero di iterazioni eseguite e termina.

Struttura

- Controllo degli argomenti (*argc*, *atoi*, valori maggiori di 0).
- *malloc* di un array *pid[]* per ricordare i PID dei figli (servono per *kill* mirata).
- Ciclo di fork. **Figlio**: libera l'array, installa l'handler di SIGUSR1 con sigaction, poi `while(1){ sleep(1); contatore++; }`. **Padre**: continua il ciclo e crea gli altri.
- Padre: `sleep(Nsec)`, poi `kill(pid[i], SIGUSR1)` per ogni figlio, poi N chiamate a *wait*.
- Handler *gestore*: stampa PID, contatore e numero del segnale, poi `exit(0)`.

Punti da sapere

- Il figlio **non deve mai uscire** dal suo *while(1)*: se lo facesse, rientrerebbe nel *for* del padre creando altri processi.
- Alternativa all'array di PID: `kill(0, SIGUSR1)`. In quel caso il padre deve **ignorare** SIGUSR1 (il default e' terminare il processo).
- Risultato atteso: ogni figlio stampa un numero di iterazioni circa uguale a Nsec.
- Il codice di uscita *exit(-3)* viene visto dal padre come 253 (valore modulo 256).

3. Esercizio 2: figliCounter

Uso: `./figliCounter <nChildren>`

Cosa fa. Ogni figlio ha un contatore e aspetta con *pause()*. Il padre, ogni 2 secondi, sceglie un figlio a caso e gli invia a caso SIGUSR1 (contatore +1) o SIGUSR2 (contatore -1). Con CTRL-C l'intero programma termina in modo ordinato.

Processo	Segnale	Effetto
Figlio	SIGUSR1	*counter*++ e stampa del nuovo valore
Figlio	SIGUSR2	*counter*-- e stampa del nuovo valore
Figlio	SIGINT	stampa il valore finale ed esegue *exit(0)*
Padre	SIGINT	esegue *wait* su tutti i figli e pone *flagTerminato* = *true*

Struttura

- **Figlio**: salva il proprio indice in *child_index*, installa i tre handler, poi `while(1) pause();`.
- **Padre**: salva i PID in *children[]*, installa l'handler di SIGINT, poi `while(!flagTerminato)` sceglie figlio e segnale con *rand()*, fa *kill* e *sleep(2)*.

Punto chiave per l'esame. Premendo CTRL-C il terminale invia SIGINT a **tutto il gruppo di processi in foreground**: lo ricevono sia il padre sia i figli. I figli stampano ed escono; il padre, nel suo handler, li aspetta con *wait* e imposta il flag. Lo *sleep(2)* del padre viene interrotto dal segnale, quindi il ciclo riparte e vede subito il flag.

Difetti noti

- *rand()* senza *srand(time(NULL))* genera sempre la stessa sequenza.
- Piccola race: il padre puo' inviare il primo segnale prima che un figlio abbia installato gli handler (con SIGUSR1 di default il figlio morirebbe).
- Le *printf* negli handler non sono async-signal-safe (in teoria andrebbe usata *write*); in laboratorio e' tollerato.

4. Esercizio 3: cercaCarFile

Uso: `./cercaCarFile C1 [C2 ... Cn] <NomeFile> <NumSec>`

Cosa fa. Una **gara**. Il padre crea un figlio per ogni carattere Ci. Ogni figlio legge il file un byte alla volta e conta le occorrenze del proprio carattere. Il primo che finisce di leggere vince e avvisa gli altri, che si fermano ("sconfitta"). Se nessuno finisce entro NumSec secondi, il padre li ferma tutti con il timeout.

Ruolo dei segnali

Segnale	Chi lo invia	Significato	Handler
SIGINT	padre -> figli	**VIA!** sblocca il *pause()* e parte il conteggio	*start_race*
SIGUSR1	vincitore -> tutti	"ho vinto"	figli: *defeat* (stampa ed esce); padre: *defeat_padre* (*termination = 1*)
SIGUSR2	padre -> figli	timeout scaduto	*timeout_handler* (stampa ed esce)

Svolgimento passo passo

1. **Parsing argomenti**: i primi *argc-3* argomenti sono caratteri singoli (*strlen == 1*), poi il nome del file (verificato con *open*), poi NumSec (controllato cifra per cifra). Il numero di processi e' *nProc = argc - 3*.
2. **Installazione degli handler prima delle fork**, cosi' i figli li ereditano.
3. **Fork**: ogni figlio apre il file, imposta *child_index* e si ferma su *pause()* in attesa del VIA.
4. **Padre**: stampa "Pronti, partenza..." e cambia le **proprie** disposizioni: ignora SIGINT e SIGUSR2, installa *defeat_padre* per SIGUSR1. Serve perche' *kill(0, ...)* colpisce anche lui.
5. `sleep(2)` di sincronizzazione (tutti i figli devono essere arrivati al *pause()*), poi `kill(0, SIGINT)`: e' il VIA.
6. Il padre esegue `sleep(timeout)`.
7. **Vincitore**: terminata la lettura del file, pone SIGUSR1 a *SIG_IGN* (per non sconfiggere se stesso), esegue `kill(0, SIGUSR1)` e stampa "Vittoria".
8. Il SIGUSR1 **interrompe lo sleep(timeout)** del padre: trova *termination == 1* e non invia SIGUSR2. Se invece nessuno ha vinto, il padre invia `kill(0, SIGUSR2)`.
9. Il padre esegue *wait* nProc volte e stampa "gara terminata regolarmente" oppure "in anticipo".

Differenza tra le due versioni

Il comportamento e' identico. Cambia solo la API: `signal(SIGINT, start_race)` nella prima; nella seconda si usa una `struct sigaction` con *sigemptyset(&sa.sa_mask)*, *sa_flags = 0* e *sa_handler*, seguita da *sigaction()*. **sigaction e' preferita** perche' il comportamento di *signal()* varia tra sistemi (reset dell'handler dopo la prima consegna, riavvio delle system call) mentre sigaction permette di controllarlo ed e' portabile.

Perche' la sleep(2) e il pause()

Se il padre inviasse SIGINT prima che un figlio sia arrivato al *pause()*, l'handler verrebbe eseguito subito e il figlio resterebbe poi bloccato per sempre. Lo *sleep(2)* e' una sincronizzazione "rozza" ma sufficiente in laboratorio; la soluzione rigorosa userebbe *sigprocmask* e *sigsuspend*.

5. Come rifare gli esercizi: metodo

Scheletro valido per tutti e tre i programmi:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
```

```
#include <sys/wait.h>

static volatile sig_atomic_t counter = 0;

void handler(int signo) { /* poche istruzioni: aggiorna variabili, stampa, exit */ }

int main(int argc, char **argv) {
    /* 1. controllo argomenti (argc, atoi > 0, file esistente...) */

    /* 2. installo gli handler PRIMA delle fork */
    struct sigaction sa;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    sa.sa_handler = handler;
    if (sigaction(SIGUSR1, &sa, NULL) == -1) { perror("sigaction"); exit(1); }

    /* 3. fork */
    for (int i = 0; i < n; i++) {
        pid_t pid = fork();
        if (pid < 0) { perror("fork"); exit(2); }
        if (pid == 0) {
            /* codice figlio: deve finire con exit() o ciclo infinito */
            exit(0);
        }
        pid_v[i] = pid;
    }

    /* 4. codice padre: kill / sleep / pause */
    /* 5. wait() n volte */
    return 0;
}
```

Ordine di ragionamento davanti a una traccia d'esame

1. Disegna il **diagramma dei messaggi**: chi manda quale segnale a chi (come la tabella dell'Es. 3).
2. Per ogni processo elenca i segnali che deve **gestire** e quelli che deve **ignorare**.
3. Chiediti: chi riceve *kill(0, ...)*? Tutto il gruppo, **te compreso**.
4. Chiediti: il figlio puo' uscire dal suo ciclo e rientrare nel *for* del padre? Non deve.
5. Chiediti: c'e' una race tra invio del segnale e installazione dell'handler o *pause()*? Se si', serve una sincronizzazione.
6. Controlla che ci siano tante chiamate a *wait* quanti sono i figli.

Compilazione e test

```
gcc -Wall -o itercounter Lab05-es01-itercounter.c
./itercounter 3 5

gcc -Wall -o figliCounter Lab05-es02-figliCounter.c
./figliCounter 3        # altro terminale: ps, kill -USR1 <pid>; poi CTRL-C

gcc -Wall -o cercaCarFile Lab05-es03-cercaCarFile-sigaction.c
./cercaCarFile a e i testo.txt 10
```

6. Checklist di ripasso

- Perche' *volatile sig_atomic_t* per le variabili toccate dagli handler.
- Perche' gli handler si installano prima della *fork* e cosa ereditano i figli.
- Cosa fa *kill(0, sig)* e perche' il padre deve ignorare o gestire quel segnale.
- Perche' *sleep* e *pause* vengono interrotti dai segnali e che conseguenze ha (es. il SIGUSR1 che anticipa la fine nel padre dell'Es. 3).
- Differenze tra *signal* e *sigaction*.
- Ruolo di *wait*: evita i processi zombie e fa terminare il padre dopo i figli.

- Perche' nell'Es. 1 e nell'Es. 3 il figlio non rientra mai nel ciclo di fork del padre.